

프로그래밍 역량 강화 전문기관, 민코딩

SSD 프로젝트 소개



목차

1. SSD 프로젝트 개요
2. SSD 제작
3. Test Shell 개발
4. Test Script 제작
5. 프로젝트 전체 일정 정리

SSD 프로젝트 개요

SSD + Test Shell 개발 Project 시작

SSD 검증용 Test Shell

SSD 제품을 테스트 할 수 있는 Test Shell 을 제작

- 실제 HW가 아닌, SW로 가상으로 구현한다.
- Test Shell 프로그램을 제작하여 SSD 동작을 테스트 할 수 있다.
- 다양한 Test Script를 제작한다.



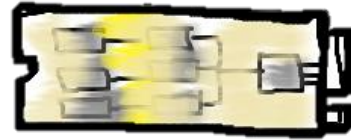
Test 수행



프로젝트 구성

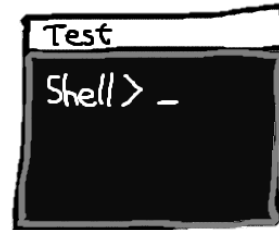
1. SSD

- HW 대신, Software로 구현한다.



2. Test Shell

- SSD를 테스트하는 프로그램



3. Test Script

- Test Shell 안에서 구현되는 SSD 테스트 코드

```
OneWRtest()  
    SSD_WRITE(0, 0xAB)  
    a = SSD_READ(0)  
    if (a == 0) print("PASS")  
    else print("FAIL")
```

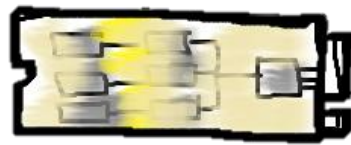
SSD 사전 지식

저장할 수 있는 공간

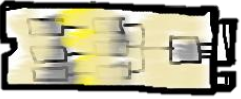
- 저장할 수 있는 최소 공간의 사이즈는 **4KB**
 - 4KB = 1 Byte 글자 기준, 총 4,096 글자 저장 가능 공간
 - 1 Byte 글자를 저장하더라도 4KB 단위로 저장된다.
- 각 공간마다 **LBA** (Logical Block Address) 라는 주소를 가짐
- OS는 FileSystem 을 거쳐 SSD에게 명령어를 전달한다.
- Read / Write / Sync / Erase 등 다양한 명령어가 존재한다.



SSD 제작



HW가 해야할 역할을 대신하여, SSD를 SW로 구현한다.



구현해야 할 가상 SSD

최소화된 기능 수행

- Read 명령어와 Write 명령어만 존재
- LBA 단위는 4 Byte
- LBA 0 ~ 99 까지 100 칸을 저장할 수 있다.

총 400 Byte를 저장 할 수 있는 가상 SSD 를 구현





Write 명령어 동작 예시 1

APP 이름 : ssd

- C++ : ssd.exe
- Java : ssd.jar
- Python : ssd.py

Write 명령어 사용 예시 1

- ssd **W** 3 0x1298CDEF : 3 번 LBA 영역에 값 0x1298CDEF 를 저장한다.

0	1	2	3	4	5	6
			1298 CDEF			

.....

96	97	98	99



Write 명령어 동작 예시 2

Write 명령어 사용 예시 2

- ssd W 2 0xAAAABBBB
- ssd W 97 0x9988FFFF

출력결과 : 없음 (저장만 수행)

0	1	2	3	4	5	6
		AAAA BBBB	1298 CDEF			

■ ■ ■ ■ ■ ■

96	97	98	99
	9988 FFFF		

```
C:\Users\minco>ssd W 2 0xAAAABBBB|
```

명령어는 CMD에서 실행할 수 있다.
한 명령어를 수행할 때 마다,
ssd 프로그램이 실행 후 종료된다.



Read 명령어 동작 예시

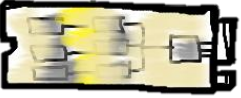
Read 명령어 사용 예시

- ssd R 2
 - 출력결과 : 0xAAAABBBB
- ssd R 97
 - 출력결과 : 0x9988FFFF

0	1	2	3	4	5	6
		AAAA BBBB	1298 CDEF			

■ ■ ■ ■ ■ ■

96	97	98	99
	9988 FFFF		



실제로 저장하는 위치

`ssd_nand.txt` 파일에 데이터를 기록한다.

- 실제 SSD는 Write 명령어를 수행할 때, SSD 내부 저장소인 Nand에 기록이 된다. 이를 모사하여, `ssd_nand.txt` 파일에 값을 저장 해 둔다.
- `ssd_nand.txt` 파일이 없는 경우, 생성 후 데이터가 기록되어야 한다.

0	1	2	3	4	5	6	...				96	97	98	99
		AAAA BBBB	1298 CDEF									9988 FFFF		



Write와 Read 명령어 상세동작

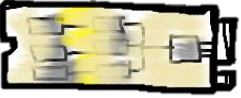
아무런 Console 출력을 하지 않는다.

- Write 명령어 수행시

- Write 명령어는 `ssd_nand.txt`에만 데이터를 기록한다.

- Read 명령어 수행시

- Read 명령어는 `ssd_nand.txt`에서 데이터를 읽고, 읽은 데이터를 `ssd_output.txt` 파일에 기록한다.
- `ssd_output.txt`에는 항상 마지막 Read 명령어에 대한 수행 결과가 저장되어있다. 즉, 덮어쓰기 방식으로 파일에 읽은 값을 기록한다.



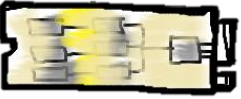
SSD 동작 예시

Read 명령 시 결과값이 **ssd_output.txt** 파일에 기록됨

Write 명령 시 **ssd_nand.txt** 파일에 값을 저장

명령어 입력순서	입력	출력
1	W 20 0x1289CDEF	-
2	R 20	0x1289CDEF
3	R 19	0x00000000
4	W 10 0xFF1100AA	
5	R 10	0xFF1100AA

기록이 안된 LBA를 읽으면
0x00000000 값이 읽힌다.



세부 규정

Read 명령어

- ssd R [LBA]
- **ssd_output.txt** 에 읽은 값이 적힌다. (기존 데이터는 사라진다.)
- 기록이 한적이 없는 LBA를 읽으면 0x00000000 으로 읽힌다.
- 잘못된 LBA 범위가 입력되면 ssd_output.txt에 "**ERROR**"가 기록된다.

Write 명령어

- ssd W [LBA] [Value]
- **ssd_nand.txt** 에 [Value]이 기록된다.
- 잘못된 LBA 범위가 입력되면 ssd_output.txt에 "**ERROR**"가 기록된다.

데이터 범위

- [LBA] : 0 ~ 99, 10진수로 입력 받음
- [Value] : 항상 0x가 붙으며 10 글자로 표기한다. (0x00000000 ~ 0xFFFFFFFF)



ssd_nand.txt 구현 방법

명령어를 사용할 때 마다, `ssd_nand.txt` 전체 데이터를 읽어온다.

Write 명령어 사용시

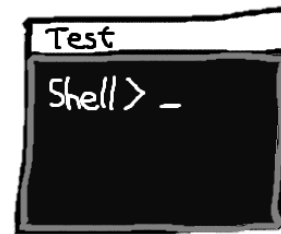
- `ssd_nand.txt` 파일 내용 전체 읽어온 후,
특정 부분을 변경하고 새로 Write를 수행한다.

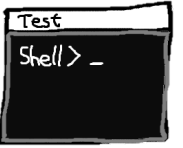
Read 명령어 사용시

- `ssd_nand.txt` 파일 내용 전체 읽어온 후,
필요한 부분을 찾아내어 반환한다.

Test Shell 개발

SSD에게 명령을 내릴 수 있는 검증용 Test Shell 제작



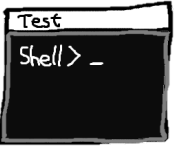


Test Shell Application

SSD를 테스트 할 수 있는 프로그램

- Shell 이 동작되어, 사용자 입력을 받는다.
- 사용 가능 명령어 종류
 - write
 - read
 - exit
 - help : 도움말
 - fullwrite : 전체 LBA 영역 Write
 - fullread : 전체 LBA 영역 Read

read / write 명령어 수행시,
제작한 "ssd" app을 실행시켜 읽기 / 저장 명령을 수행



Test Shell 동작 예시

사용자 입력 예시

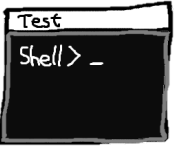
- write 3 0xAAAABBBB
 - 3번 LBA 에 0xAAAABBBB 를 기록한다.
 - 제작한 ssd 프로그램을 실행시켜 동작시킨다.
- read 3
 - 3번 LBA 를 읽는다.
 - 제작한 ssd 프로그램을 실행시켜 동작시킨다.
 - 읽은 결과를 화면에 출력한다.
 - 출력 형태는 자유롭게 결정한다.

```
Shell> write 3 0xAAAABBBB
[Write] Done

Shell> read 0
[Read] LBA 00 : 0x00000000

Shell> read 3
[Read] LBA 03 : 0xAAAABBBB

Shell> |
```



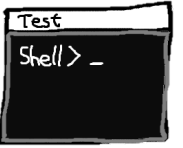
exit / help

exit 명령어

- Test Shell 이 종료된다.

help 명령어

- 제작자를 적는다. (팀 명과 팀원 이름을 적는다.)
- 각 명령어 마다 사용 방법을 출력한다.



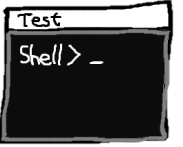
fullwrite / fullread

fullwrite 명령어

- 모든 LBA 영역에 대해 Write를 수행한다.
- 동작 예시) fullwrite 0xABCDFFFF
 - 모든 LBA에 값 0xABCDFFFF 가 적힌다.

fullread 명령어

- LBA 0 번부터 99 번 까지 Read를 수행한다.
- ssd 전체 값을 모두 화면에 출력한다.
- 동작 예시) fullread
 - 모든 LBA의 값들이 화면에 출력된다.



유의사항

기능 구현시 유의사항

- **입력받은 매개변수가 유효성 검사 수행**
 - 파라미터의 Format이 정확해야 함
 - LBA 범위는 0 ~ 99
 - A ~ F, 0 ~ 9 까지 숫자 범위만 허용
- **없는 명령어를 수행하는 경우 "INVALID COMMAND" 을 출력**
 - 어떠한 명령어를 입력하더라도 Runtime Error가 나오면 안된다.

Test Script

제작한 Test Shell 안에서 동작되는 Test Script 제작하기

```
OneWRtest()  
    SSD_WRITE(0, 0xAB)  
    a = SSD_READ(0)  
    if (a == 0) print("PASS")  
    else print("FAIL")
```

```
OneWRtest()  
SSD_WRITE(0, 0xAB)  
a = SSD_READ(0)  
if (a == 0) print("PASS")  
else print("FAIL")
```

Test Scenario 란?

검증팀에서는 SSD가 모든 상황에서도 정상 동작되는지 테스트를 해야한다.
이를 위해 **검증 계획**을 세우는데, 이를 **Test Scenario** 라고 한다.
주로 엑셀에다가 Test Scenario를 작성하곤 한다.

Test Scenario 예시

Test Scenario 1번

Full Write 한 후, Full Read 100회 를 했을 때,
100회 모두 같은 값이 올라오는지 확인하는 테스트

Test Scenario 2번

Full Write 를 3번 한 후, Full Read를 1회 해서
가장 마지막에 Write한 값이 올라오는지 테스트




```
OneWRtest()  
    SSD_WRITE(0, 0xAB)  
    a = SSD_READ(0)  
    if (a == 0) print("PASS")  
    else print("FAIL")
```

Test Script란?

계획을 세운 Test Scenarios대로 검증할 수 있는 코드를 작성해야한다!

이를 **Test Script** 라고 한다.

```
OneWRtest()
SSD_WRITE(0, 0xAB)
a = SSD_READ(0)
if (a == 0) print("PASS")
else print("FAIL")
```

ReadCompare

Test Script는 아래와 같은 방법으로 검증한다.

Unit Test에서는 기댓값과 실제값을 비교하는 동작을 Assertion 이라고 했었다.

우리 Test Script 프로젝트에서는 "**ReadCompare**" 동작 이라고 하겠음 (비공식 용어)

Loop {

1. 값 Write

2. **ReadCompare**

--> Read 명령어 수행 후,
Write 할때 넘긴 값, 실제로 Read된 값이 일치되는지 확인

}

```
OneWRtest()  
  SSD_WRITE(0, 0xAB)  
  a = SSD_READ(0)  
  if (a == 0) print("PASS")  
  else print("FAIL")
```

PASS / FAIL 기준

Read Compare에서 FAIL 여부가 결정된다.

- **Read Compare** : Read 명령어 수행 후, 기댓값과 일치하는지 확인
- FAIL이 발생하면 즉시 Test Script가 종료되며, 화면에 **FAIL**을 출력한다.
- FAIL이 발생된적이 없이 Test Script가 종료되면, 화면에 PASS를 출력한다.

```
Loop {
```

```
  1. Write [LBA 0], [값 0xA]
```

```
  2. if ReadCompare([LBA 0], [0xA]) == False : return FAIL
```

```
}
```

```
return PASS
```

```
OneWRtest()  
SSD_WRITE(0, 0xAB)  
a = SSD_READ(0)  
if (a == 0) print("PASS")  
else print("FAIL")
```

제작할 Test Script 1

Full Write 후 ReadCompare 수행

- Test Script 이름 : 1_FullWriteAndReadCompare

Test Script 실행 방법

- Test Shell 에서 "1_FullWriteAndReadCompare" 라고 입력한다.
- Test Shell 에서 "1_" 만 입력해도 실행 가능하다.

Test Scenario

- 0 ~ 4번 LBA까지 Write 명령어를 수행한다.
- 0 ~ 4번 LBA까지 ReadCompare 수행
- 5 ~ 9번 LBA까지 다른 값으로 Write 명령어를 수행한다.
- 5 ~ 9번 LBA까지 ReadCompare 수행
- 10 ~ 14번 LBA까지 다른 값으로 Write 명령어를 수행한다.
- 10 ~ 14번 LBA까지 ReadCompare 수행
- 위와 같은 규칙으로 전체 영역에 대해 Full Write + Read Compare를 수행한다.

```
OneWRtest()  
SSD_WRITE(0, 0xAB)  
a = SSD_READ(0)  
if (a == 0) print("PASS")  
else print("FAIL")
```

제작할 Test Script 2

LBA 순서를 섞어가며 Write 수행

- Test Script 이름 : 2_PartialLBWrite

Test Script 실행 방법

- Test Shell 에서 "2_PartialLBWrite" 라고 입력한다.
- Test Shell 에서 "2_" 만 입력해도 실행 가능하다.

Test Scenario

- Loop는 30회
 - 4번 LBA에 값을 적는다.
 - 0번 LBA에 같은 값을 적는다.
 - 3번 LBA에 같은 값을 적는다.
 - 1번 LBA에 같은 값을 적는다.
 - 2번 LBA에 같은 값을 적는다.
 - LBA 0 ~ 4번, ReadCompare

```
OneWRtest()  
    SSD_WRITE(0, 0xAB)  
    a = SSD_READ(0)  
    if (a == 0) print("PASS")  
    else print("FAIL")
```

제작할 Test Script 3

Write Read 반복 테스트

- Test Script 이름 : 3_WriteReadAging

Test Script 실행 방법

- Test Shell 에서 "3_WriteReadAging" 라고 입력한다.
- Test Shell 에서 "3_" 만 입력해도 실행 가능하다.

Test Scenario

- Loop 200회
 - 0번 LBA에 랜덤 값을 적는다.
 - 99번 LBA에 랜덤 값을 적는다.
 - LBA 0번과 99번, ReadCompare를 수행

팀 프로젝트 진행

1 ~ 2일차 / 3 ~ 5 일차 프로젝트 진행 안내

프로젝트 준비

- ✓ 창의력있는 **팀 이름**을 정해주세요.
- ✓ 팀에서 사용할 코딩규칙 (코드 컨벤션, 코딩가이드룰)을 설정합니다.
- ✓ 팀장님과 팀원 분들의 역할을 분배합니다

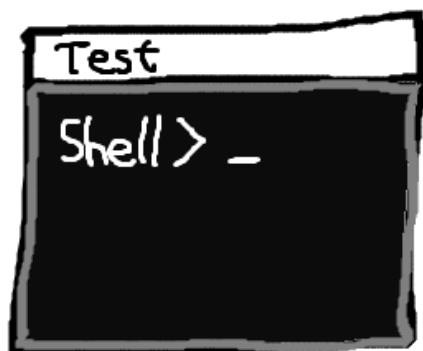
[1~2일차] 요청사항

✓ 1 ~ 2일차 개발시 **Test Double**을 한번 이상 사용해주세요.

- SSD 성능이 느리기에 Unit Test가 오래걸리는 이슈가 있습니다
- Test Double로 이러한 문제를 해결할 수 있습니다.

✓ **TDD** 개발 필수

- 1일차 ~ 2일차 : TDD 개발이 포함되어야 합니다.
- 3일차 ~ 4일차 : TDD 없이 개발해도 됩니다. Unit Test + 리팩토링만 해주세요.



Test Shell
(with Test Script)

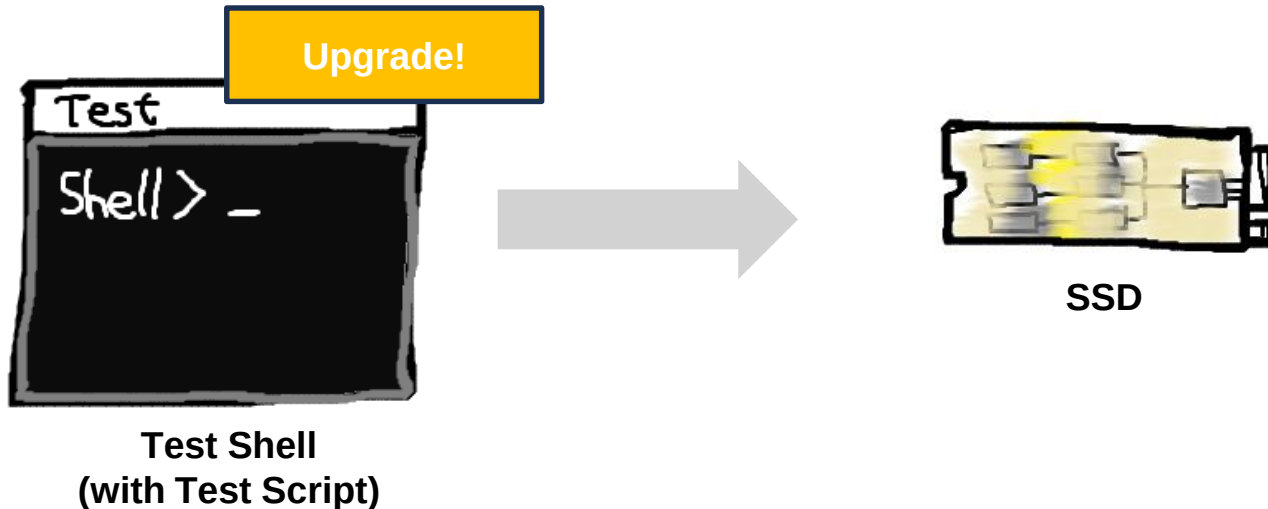


SSD

[3~4일차] 기능 추가 개발 및 리팩토링

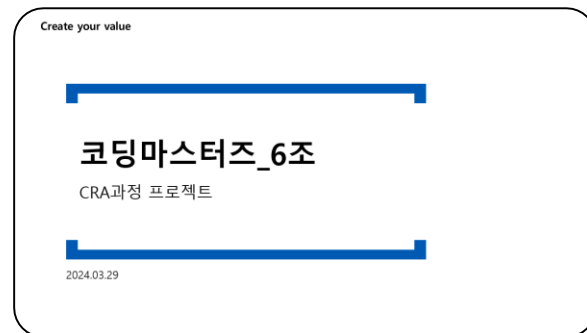
✓ 3일차에는 기능 추가를 요청을 드릴 예정입니다.

- 명령어 추가 / 성능 최적화 / 로그 관리 기능 등, 기능 추가 요청 드립니다.
- 기능 추가를 대비하여, 1 ~ 2일차 까지 클린한 코드를 만들어두어야 합니다.



[5일차] 발표 준비 및 발표

- ✓ 5일차에는 그 동안 제작했던 프로젝트 발표 준비를 합니다.
 - ✓ 오전 : PPT 준비 / 오후 : 발표 및 마무리
 - ✓ 팀장님이 발표해주시면 됩니다. (변경하셔도 됩니다.)
- ✓ 발표 자료에는 리팩토링 전 후 결과와 코드리뷰활동 캡처가 포함되어야합니다.
 - ✓ 클린코드를 신경쓰지 않는 코드를 먼저 만들고, 리팩토링을 통해 점차 개선하는 방식으로 개발을 권장합니다.
 - ✓ 리팩토링 전 후 결과 캡처가 가능하도록, 리팩토링 과정마다 Commit을 권장합니다.



[참고] 팀 프로젝트 일정

	팀 프로젝트								
08:30 – 09:30	수평적 코드 리뷰를 위한 팀 빌딩 - 커뮤니케이션	Team Project 미션1 수행	Team Project 미션 2 안내 Team Project 미션 2 안내	Team Project 미션 2 수행	Team Project 마무리 및 발표준비				
09:30 – 10:30									
10:30 – 11:30									
11:30 – 12:30									
12:30 – 13:30	Team Project 개발환경 세팅 Team Project 미션1 안내	Team Project 미션1 수행	Team Project 미션 2 수행	Team Project 미션 2 수행	Team Project 발표				
13:30 – 14:30									
14:30 – 15:30	Team Project 미션1 수행								
15:30 – 16:30									
16:30 – 17:30					사내 현업활동 가이드				

SSD 프로젝트 시작

✓ SSD 프로젝트를 시작해주세요

- 프로젝트 진행 / 기능 구현 / 디버깅 질문
프로젝트 코치에게 언제든지 질문주세요.

- 1주차 ~ 2주차 수업 내용 관련 질문

- 프로젝트 코치에게 Discord로 DM 주시면, 담당 강사에게 전달드리겠습니다.
- 최인호 강사에게 직접 질문하기 : inho.choi@mincoding.co.kr
- 서정환 강사에게 직접 질문하기 : jeonghwan.seo@mincoding.co.kr

0	1	2	3	4	5	6
			1298 CDEF			

